

Riferimento tecnico

TDS Toolkit

Scrittura di un decodificatore di protocollo

Per gli oscilloscopi digitalizzatori delle serie TDS 500, TDS 600 e TDS 700

Documento TDSTK-DEC-01

Copyright © 2026 Jared Cabot. Licenza MIT.

TDSTK-DEC-01

Copyright © 2026 Jared Cabot. Tutti i diritti riservati.

Questo documento descrive l'interfaccia dei plug-in di decodifica di TDS Toolkit. Le informazioni contenute in questa pubblicazione sostituiscono quelle di tutto il materiale precedentemente pubblicato.

TDS Toolkit è software host; non è un prodotto Tektronix. Tektronix e TDS sono utilizzati solo per denominare gli strumenti che il software supporta.

Questo software è rilasciato sotto la Licenza MIT. IL SOFTWARE VIENE FORNITO "COSÌ COM'È", SENZA GARANZIA DI ALCUN TIPO, ESPRESSA O IMPLICITA. Il testo completo accompagna il programma come LICENSE.

Indice

Manuali correlati	iv
1 Il modello del decodificatore	1-1
Che cos'è un decodificatore	1-1
Un file, un protocollo	1-1
Il contratto	1-1
Rilevamento	1-1
Dove risiedono i file	1-1
2 Il Signal	2-1
Cosa riceve una decodifica	2-1
Campioni e temporizzazione	2-1
Livelli, non campioni	2-1
Da dove proviene un Signal	2-2
3 Il contratto del decodificatore	3-1
Attributi di classe	3-1
I quattro attributi	3-1
Ruoli e sorgenti	3-1
I metodi	3-2
decode	3-2
Parametri	3-3
Il tipo Param	3-3
4 Frame e rappresentazione	4-1
Il Frame	4-1
Un singolo evento decodificato	4-1
Stato	4-1
Rappresentazione del risultato	4-2
Quattro piccoli metodi	4-2
Colore	4-2

5	Costruzione di un decodificatore	5-1
	Un esempio pratico	5-1
	Il protocollo	5-1
	I passaggi	5-1
	Il codice	5-1
	Import e attributi	5-1
	Recupero dei bit	5-2
	Raccolta delle parole	5-2
	Rappresentazione	5-3
	La scorciatoia seriale	5-4
	Non reimplementare il campionamento dei bit	5-4
6	Installazione e test	6-1
	Installazione	6-1
	Rilasciare e aprire	6-1
	Test	6-1
	Pilotarlo con un Signal sintetico	6-1
	Testare ogni decodificatore	6-1
	Lista di controllo	6-2
	Prima di farvi affidamento	6-2

Elenco delle tabelle

Tabella 1–1: File del sottosistema di decodifica	1–2
Tabella 2–1: Membri di un Signal	2–2
Tabella 3–1: Attributi di classe del Decoder	3–1
Tabella 3–2: Metodi del Decoder	3–2
Tabella 3–3: Generi di Param	3–3
Tabella 4–1: Campi del Frame	4–1
Tabella 4–2: Bit di stato del Frame	4–2

Prefazione

Questo manuale descrive come aggiungere un decodificatore di protocollo a TDS Toolkit. Un decodificatore è un unico file Python. Legge una forma d'onda catturata e restituisce un elenco di eventi decodificati; il programma lo elenca, mappa i suoi ingressi, lo esegue e disegna il risultato senza alcun ulteriore codice.

Si presuppone che il lettore sia un programmatore esperto a suo agio con Python. Non è necessaria alcuna conoscenza del resto del programma: l'intera interfaccia è una sola classe base e tre piccoli tipi di dati, tutti nel modulo `tds_decode.py`.

Leggere questo manuale accanto a due file forniti con il programma: `decoders/_template.py`, lo scheletro da copiare, e `decoders/README.txt`, la forma breve di quanto segue.

Manuali correlati

L'interfaccia, e i file da leggere accanto a questo manuale:

- *decoders/_template.py* - lo scheletro da copiare.
- *decoders/README.txt* - la forma breve di quanto segue.
- *tds_decode.py* - il motore: la classe base `Decoder` e i tipi `Signal`, `Frame` e `Param`.

Il modello del decodificatore

Che cos'è un decodificatore

Un file, un protocollo

Un decodificatore è un unico file Python nella cartella `decoders` che definisce una sottoclasse di `Decoder`. Ogni file è un protocollo. La scheda Forme d'onda elenca ogni decodificatore che trova, quindi aggiungere un protocollo è un solo file e nient'altro nel programma cambia.

Una decodifica viene eseguita interamente su una forma d'onda catturata che la scheda Forme d'onda già contiene: un canale letto dallo strumento, o un riferimento memorizzato caricato da un file. Nulla in un decodificatore tocca lo strumento. È questo il senso del progetto. Funziona su qualsiasi TDS, compresi i modelli base il cui firmware non può eseguire un'applicazione di decodifica del bus propria, perché la decodifica avviene nell'host, sui campioni, dopo la cattura.

L'intera interfaccia è la classe base `Decoder`. Un decodificatore imposta quattro attributi di classe e implementa un metodo. Gli altri tre metodi hanno impostazioni predefinite funzionanti, sovrascritte solo per rappresentare i campi propri del protocollo nella tabella e sul grafico. I tre tipi di dati che un decodificatore gestisce, `Signal`, `Frame` e `Param`, sono descritti nei capitoli che seguono.

Il contratto

All'avvio, e ogni volta che la scheda Forme d'onda viene aperta, il programma esamina la cartella `decoders` alla ricerca di file `*.py`, importa ciascuno in isolamento, e istanzia ogni sottoclasse di `Decoder` che trova. Un file il cui nome inizia con un trattino basso viene trattato come ausiliario e saltato, ed è così che `_template.py` resta un modello e non compare mai come protocollo vuoto nell'elenco.

Rilevamento

L'importazione è difensiva. Un file che non si importa, che sia per un errore di sintassi o per un riferimento errato, viene saltato, con una nota scritta nel registro del programma, anziché far cadere con sé l'intero elenco. Un decodificatore lasciato a metà non nasconde mai quelli funzionanti. Quando due cartelle offrono un decodificatore con lo stesso NAME, prevale la cartella successiva, così una copia scritta localmente ha la precedenza su quella fornita.

Dove risiedono i file

I decodificatori forniti con il programma risiedono in una cartella `decoders` all'interno dell'installazione. Una copia scrivibile è conservata accanto al file delle impostazioni, `tdstoolkit.json`, e viene reintegrata dal set fornito a ogni esecuzione, esattamente come la libreria di maschere. Un decodificatore aggiunto in una build successiva viene copiato senza sovrascrivere quello scritto localmente; un decodificatore fornito eliminato ricompare all'esecuzione successiva, perché il set fornito deve essere presente. Rilasciando un nuovo file nella cartella scrivibile, esso compare nell'elenco alla successiva apertura della scheda. Non è necessaria alcuna ricompilazione.

Tabella 1-1: File del sottosistema di decodifica

File	Ruolo
tds_decode.py	Il motore: la classe base Decoder, i tipi Signal, Frame e Param, le costanti di stato, la funzione ausiliaria seriale condivisa e il rilevamento. Un decodificatore importa da esso e non lo modifica.
decoders/_template.py	Lo scheletro. Copiarlo per iniziare un nuovo decodificatore.
decoders/README.txt	Le istruzioni in forma breve e una mappa dei decodificatori di esempio.
decoders/.py	Un protocollo ciascuno: uart.py, i2c.py, spi.py, can.py e gli altri. Leggere quello più vicino al protocollo che si sta aggiungendo.

Il Signal

Cosa riceve una decodifica

Campioni e temporizzazione

Al metodo `decode()` viene passato un `Signal` per ciascun ingresso di cui il protocollo ha bisogno. Un `Signal` è un record catturato ridotto a ciò che un decodificatore desidera: i campioni interi, e la temporizzazione che trasforma un indice di campione in un tempo. Non porta con sé volt né scalatura di visualizzazione. Un decodificatore confronta un campione con una soglia e non ha mai bisogno di conoscere la tensione, quindi i conteggi grezzi del digitalizzatore vengono trasmessi senza scalatura e un record da un quarto di milione di punti non costa nulla da consegnare.

La soglia di decisione, la banda morta di isteresi e l'ampiezza del segnale vengono calcolate prima che `decode()` venga chiamato. Un decodificatore non calcola una soglia propria. La soglia è il punto medio degli estremi del record; l'isteresi è un ottavo dell'ampiezza, abbastanza larga da respingere il ringing dei fronti senza che un fronte lento manchi di attraversarla. Questi valori si leggono, non si impostano.

Livelli, non campioni

Per la maggior parte dei protocolli la forma utile del record non sono i campioni grezzi ma un livello logico pulito per campione. Il metodo `digitize()` restituisce esattamente questo: uno 0 o un 1 per ogni campione, con l'isteresi propria del record applicata affinché il ringing dei fronti non venga letto come più attraversamenti. Chiamarlo una volta e percorrere il risultato. Un protocollo che ha bisogno dei conteggi grezzi, per esempio una linea bipolare con due soglie, legge invece `sig.data` direttamente.

Tabella 2-1: Membri di un Signal

Membro	Significato
sig.data	I campioni interi, una sequenza indicizzabile di conteggi grezzi del digitalizzatore.
sig.n	Il numero di campioni, lo stesso di len(sig.data).
sig.dt	Secondi per campione.
sig.t0	Il tempo del primo campione, in secondi relativi al trigger.
sig.first_index	L'offset nel record del primo campione. Sommarlo a un indice di campione locale per ottenere una posizione assoluta nel record.
sig.threshold	Il livello di decisione, in conteggi. Impostato dal framework.
sig.hysteresis	La banda morta attorno alla soglia, in conteggi. Impostata dal framework.
sig.span	La differenza tra il campione più alto e quello più basso del record. Impostata dal framework; un'ampiezza prossima allo zero indica una sorgente piatta, priva di segnale.
sig.time_at(i)	Il tempo in secondi all'indice di campione locale i, cioè $t_0 + i \times dt$.
sig.digitize()	Un nuovo bytearray con un livello 0/1 per campione, con isteresi.
sig.auto_threshold()	Ricalcola soglia, isteresi e ampiezza. Già chiamato dal framework; chiamato solo su un Signal costruito a mano (vedere la Sezione 6).

NOTA. Un Signal porta conteggi grezzi, non volt. Confrontare con sig.threshold, mai con una tensione. Un decodificatore che legge correttamente a una scala verticale legge poi correttamente a ogni altra scala, perché la soglia si sposta con il record.

Da dove proviene un Signal

Il Signal che decode() riceve è costruito dal programma, non dal decodificatore, a partire dalla forma d'onda catturata con il metodo di classe Signal.from_waveform(). Legge i livelli interi e l'intervallo di campionamento e l'offset del trigger dal preambolo della forma d'onda, costruisce il Signal e chiama auto_threshold() su di esso prima di consegnarlo al decodificatore. Un Signal viene costruito a mano solo quando si scrive un test, argomento trattato nella Sezione 6.

Il contratto del decodificatore

Attributi di classe

Una sottoclasse di decodificatore imposta quattro attributi di classe. NAME e DESCRIPTION lo denominano nell'elenco dei protocolli. SOURCES dichiara gli ingressi di forma d'onda di cui ha bisogno. PARAMS dichiara le impostazioni che la scheda offre. Tutto il resto è ereditato.

Tabella 3-1: Attributi di classe del Decoder

Attributo	Significato
NAME	Il nome del protocollo. Mostrato nell'elenco e usato come titolo dell'etichetta del grafico. Lasciandolo vuoto, o "?", il caricatore non elenca la classe; è così che il modello si nasconde.
DESCRIPTION	Una riga per la seconda colonna dell'elenco.
SOURCES	Un elenco di nomi di ruolo, in ordine. Un solo ruolo è un protocollo a un filo; due o più ricevono ciascuno una sorgente catturata mappata su di essi. Un "?" finale contrassegna un ruolo facoltativo che può essere lasciato non mappato.
PARAMS	Un elenco di oggetti Param, un controllo ciascuno. Può essere vuoto.
COLOUR	Il colore del chevron per un Frame pulito, come "#rrggbb". Un decodificatore possiede i propri colori; l'app non ha alcuna impostazione di colore per i decodificatori. Impostarlo per un unico colore per l'intero protocollo.
FAULT_COLOUR	Il colore del chevron per un Frame in errore. Rosso per impostazione predefinita, così che un errore risalti su qualsiasi colore di protocollo.

Ruoli e sorgenti

SOURCES è sia il numero di forme d'onda di cui un protocollo ha bisogno sia i loro nomi. Un protocollo a un filo dichiara un solo ruolo, per convenzione "Line". Un protocollo a due fili come I2C dichiara ["SDA", "SCL"]; la scheda mostra un selettore di sorgente per ruolo, e decode() riceve un Signal per ciascuno. Un punto interrogativo finale, come in ["CLK", "MOSI", "MISO?", "CS?"] di SPI, contrassegna un ruolo che l'operatore può lasciare non mappato; il suo Signal è allora assente dal dizionario che decode() riceve, e il codice di decode() lo verifica.

I metodi

decode `decode(self, signals, params)` è l'unico metodo obbligatorio. `signals` è un dizionario che associa ciascun nome di ruolo, senza alcun "?" finale, al suo Signal; un ruolo facoltativo non mappato è assente. `params` è un dizionario che associa ciascuna chiave di Param al valore scelto dall'operatore. Il metodo restituisce una tupla di due elementi: un elenco di oggetti Frame e una stringa di nota di una riga.

La nota è mostrata sopra la tabella dei risultati. Usarla per riportare ciò che la decodifica ha dedotto, come la velocità in bit rilevata, i campioni per bit e il numero di Frame trovati, oppure, quando il record non può essere letto, per dirne il motivo. In caso di errore, restituire un elenco di Frame vuoto e una nota che spieghi il problema in una frase.

Tabella 3-2: Metodi del Decoder

Metodo	Restituisce
<code>decode(signals, params)</code>	<code>(frames, note)</code> . La decodifica stessa. Obbligatorio.
<code>columns()</code>	Le intestazioni della tabella dopo la colonna Time, come elenco di coppie <code>(heading, width_in_pixels)</code> . Sovrascrivere per denominare i campi del protocollo.
<code>row(frame, ascii)</code>	Le celle della tabella per un Frame, corrispondenti a <code>columns()</code> dopo il tempo. Sovrascrivere per formattare i campi del protocollo.
<code>label(frame, ascii)</code>	La stringa utile più breve per il chevron sul grafico. Sovrascrivere per mantenerla concisa.
<code>colour(frame)</code>	Il colore del chevron per un Frame. Predefinito a COLOUR per un Frame pulito e a FAULT_COLOUR per uno in errore. Sovrascrivere per colorare per singolo chevron, discriminando su <code>frame.kind</code> .

L'argomento `ascii` passato a `row()` e `label()` è lo stato dell'interruttore Hex / ASCII della scheda. Rispettarlo dove ha senso, un byte mostrato come carattere o come due cifre esadecimali, e ignorarlo dove non ne ha.

Parametri

Ogni Param nell'elenco PARAMS diventa un controllo sulla scheda, così un protocollo aggiunge le proprie opzioni senza alcun codice di interfaccia utente. Il costruttore è `Param(key, label, kind, choices, default, lo, hi, unit, note)`. `key` è la chiave del dizionario che `decode()` legge; `label` è la didascalia che l'operatore vede; `kind` è una di tre stringhe.

Tabella 3–3: Generi di Param

kind	Controllo e campi
"choice"	Un menu a discesa. <code>choices</code> è un elenco di coppie (<code>value</code> , <code>label</code>); il <code>value</code> è ciò che raggiunge <code>params</code> , la <code>label</code> è ciò che viene mostrato. <code>default</code> è uno dei <code>value</code> .
"int"	Un campo di immissione di interi. <code>lo</code> e <code>hi</code> lo delimitano; <code>unit</code> è un suffisso della didascalia come "bits". Un <code>default</code> di 0 è per convenzione inteso come "auto", cosa che la nota dovrebbe indicare.
"bool"	Una casella di spunta. <code>default</code> è <code>True</code> o <code>False</code> .

Il tipo Param

Dove `default` è omissso, ricade sulla prima scelta per un `choice`, su `False` per un `bool` e su 0 per un `int`. La nota facoltativa è mostrata accanto al controllo come suggerimento.

Frame e rappresentazione

Il Frame

Un singolo evento decodificato

Un Frame è un singolo evento decodificato: un carattere UART, un byte I2C, una parola SPI, una condizione di bus. Il tipo è piatto e condiviso da ogni protocollo. Porta con sé un dato primario, uno secondario facoltativo, una length, uno status e un kind. Cosa significhino i campi è competenza del decodificatore, poiché solo il decodificatore che ha creato un Frame lo rappresenta, ma la loro forma è fissa. Costruire un Frame con argomenti keyword; ogni campo ha un valore predefinito. I due campi di indice contano di più e sono i più facili da sbagliare. `index` e `end_index` sono posizioni assolute di campione nel record: il grafico disegna il riquadro del chevron dall'uno all'altro. Formarli come `sig.first_index` più l'indice di campione locale, esattamente come il tempo proviene da `sig.time_at()` dell'indice locale.

Tabella 4-1: Campi del Frame

Campo	Significato
time	Il tempo di inizio dell'evento in secondi, da <code>sig.time_at(local_index)</code> .
index	Il campione di inizio assoluto: <code>sig.first_index</code> + inizio locale.
end_index	Il campione di fine assoluto: <code>sig.first_index</code> + fine locale. Il riquadro va da <code>index</code> a <code>end_index</code> .
value	Il dato primario: il byte, la parola, il valore misurato.
tag	Un dato secondario facoltativo: un indirizzo I2C, una parola MISO SPI, una frequenza misurata. 0 se non utilizzato.
length	La lunghezza del dato, nell'unità in cui il protocollo conta. Predefinito a 1.
status	Un OR bit a bit delle costanti di stato; OK (zero) quando il Frame è pulito.
kind	Una stringa libera, come "byte", "addr", "data", "start" o "stop", su cui <code>row()</code> e <code>label()</code> discriminano per rappresentare ciascuna parte di un Frame in modo distinto.

Stato

`status` è una maschera di bit. Un Frame pulito è OK, cioè zero. Un Frame con problemi combina in OR una o più delle costanti nella Tabella 4-2. Il framework disegna qualsiasi Frame il cui `status` è diverso da zero come un errore, un chevron rosso e un punto esclamativo iniziale nell'etichetta, senza che il decodificatore faccia nulla oltre a impostare il bit.

Tabella 4-2: Bit di stato del Frame

Costante	Valore	Significato
OK	0	Pulito; nessun errore.
FRAMING_ERROR	1	Il Frame non si è chiuso come il suo formato richiede, come un bit di stop UART che non era un mark.
PARITY_ERROR	2	Un bit di parità non concordava con i dati.
TRUNCATED	4	Il record è terminato a metà del Frame.
ACK_ERROR	8	Un byte che si attendeva un riconoscimento non è stato riconosciuto.

Importare le costanti usate da `tds_decode` insieme a `Decoder`, `Frame` e `Param`. Non definire valori di status oltre a questi; i cinque sopra sono ciò che il `render` comprende.

Rappresentazione del risultato

Quattro piccoli metodi

La classe base rappresenta un Frame come un semplice byte: hex, ASCII e una parola di stato. Sovrascrivere `columns()`, `row()` e `label()` per parlare invece il vocabolario del protocollo. `columns()` restituisce le intestazioni dopo la colonna Time, ciascuna con una larghezza in pixel. `row()` restituisce una cella per intestazione, nello stesso ordine. `label()` restituisce la stringa utile più breve per il chevron disegnato sulla forma d'onda stessa.

Un protocollo i cui Frame portano kind diversi discrimina su `frame.kind` in questi metodi, così che uno START, un byte di indirizzo e un byte di dati si leggano ciascuno per ciò che è nella tabella e sul grafico. Mantenere `label()` concisa: viene disegnata dentro un piccolo riquadro sopra la traccia, quindi da uno a tre caratteri è la lunghezza giusta.

Colore

Il quarto metodo, `colour(frame)`, imposta il colore del chevron, e il decodificatore lo possiede completamente: l'app non ha alcuna impostazione di colore per i decodificatori. L'impostazione predefinita colora un Frame pulito con il `COLOUR` della classe e uno in errore con `FAULT_COLOUR`, così impostare `COLOUR` dà all'intero protocollo un unico colore. Sovrascrivere `colour()` e discriminare su `frame.kind` per applicare un colore per chevron, così che l'indirizzo, i dati e il framing di una stessa linea si leggano ciascuno in un colore proprio. Restituire un `"#rrggbb"`; ricadere sulla classe base per i kind che il decodificatore non colora da sé.

Costruzione di un decodificatore

Un esempio pratico Il protocollo

Questo capitolo costruisce un decodificatore completo a partire dal modello. Il protocollo d'esempio è un unico filo autotemporizzato che porta un bit per impulso: la linea sale, resta alta per una frazione del periodo di bit e scende; un breve stato alto è uno 0 e un lungo stato alto è un 1, deciso dal fatto che il tempo di stato alto sia più della metà del periodo. I bit vengono raccolti in parole di dimensione e ordine dei bit scelti. È la famiglia della linea di pixel WS2812 e del telecomando a infrarossi NEC, e non è tra i protocolli integrati, quindi è un decodificatore realmente nuovo anziché la copia di uno esistente.

I passaggi

1. *Copiare il modello:* Copiare `decoders/_template.py` in `decoders/pulsebits.py`.
2. *Denominare il protocollo:* Rinominare la classe e impostare `NAME`, `DESCRIPTION` e `SOURCES`.
3. *Dichiarare le impostazioni:* Riempire `PARAMS` con i controlli per la dimensione della parola e l'ordine dei bit.
4. *Scrivere la decodifica:* Recuperare i bit dalla traccia dei livelli, poi raccoglierli in parole.
5. *Rappresentare i campi:* Sovrascrivere `columns()`, `row()` e `label()` per mostrare la parola in hex e binario.
6. *Testarlo:* Pilotare il decodificatore con un Signal sintetico e verificare il risultato, come mostra la Sezione 6.

Il codice

Import e attributi

L'intestazione importa solo da `tds_decode`, denomina la classe e dichiara i due ingressi che la scheda offrirà: una dimensione della parola e un ordine dei bit. `SOURCES` contiene un solo ruolo, quindi la scheda mostra un unico selettore di sorgente.

```
from tds_decode import Decoder, Frame, Param
from tds_decode import OK, TRUNCATED

class PulseBitsDecoder(Decoder):

    NAME = "Pulse-width bits"
    DESCRIPTION = "Self-clocked pulse-width line, one wire"

    SOURCES = ["Line"]
    PARAMS = [
        Param("bits", "Word size", "int", default=8,
              lo=1, hi=32, unit="bits"),
        Param("order", "Bit order", "choice",
              choices=[("msb", "MSB first"),
                       ("lsb", "LSB first")],
              default="msb"),
    ]
```

Recupero dei bit

decode() prende l'unico Signal, si protegge da una sorgente piatta e riduce il record a un livello 0/1 per campione con digitize(). Percorre poi la traccia dei livelli. Ogni bit inizia su un fronte di salita; la sequenza di campioni alti è l'impulso, la sequenza di campioni bassi che segue prosegue fino al fronte di salita successivo, e le due insieme sono il periodo di bit. Il tempo di stato alto rispetto al periodo decide il bit.

```
def decode(self, signals, params):
    sig = signals["Line"]
    if sig.span < 8:
        return [], "no signal on this source"
    level = sig.digitize()
    n = len(level)
    word_bits = max(1, int(params.get("bits") or 8))
    msb = params.get("order", "msb") == "msb"

    bits = [] # (value, start_index, end_index)
    i = 1
    while i < n:
        if level[i] and not level[i - 1]: # a rising edge
            start = i
            while i < n and level[i]: # the high pulse
                i += 1
            high = i - start
            while i < n and not level[i]: # low, to next rise
                i += 1
            period = i - start
            if period > 0:
                bit = 1 if high * 2 > period else 0
                bits.append((bit, start, i))
        else:
            i += 1
```

Raccolta delle parole

I bit vengono poi presi in gruppi di `word_bits`, con il più o il meno significativo per primo a seconda di quanto dice il parametro. Un gruppo che risulta incompleto perché il record è terminato viene comunque emesso, con il suo status contrassegnato `TRUNCATED` così che il renderer lo disegni come un errore. Ogni Frame prende il suo tempo dal primo bit e si estende dall'inizio del primo bit alla fine dell'ultimo. La nota riporta i totali.

```
frames = []
for base in range(0, len(bits), word_bits):
    chunk = bits[base:base + word_bits]
    value = 0
    for pos, (bit, _s, _e) in enumerate(chunk):
        shift = (word_bits - 1 - pos) if msb else pos
        value |= bit << shift
    start_local = chunk[0][1]
    end_local = chunk[-1][2]
    ok = len(chunk) == word_bits
    frames.append(Frame(
        time=sig.time_at(start_local),
        index=sig.first_index + start_local,
        end_index=sig.first_index + end_local,
        value=value, length=len(chunk),
        status=OK if ok else TRUNCATED,
        kind="data"))
return frames, "%d bit(s), %d word(s)" % (
    len(bits), len(frames))
```

Rappresentazione

I tre metodi di rappresentazione mostrano la parola come hex e come binario della sua stessa larghezza in bit, e contrassegnano una parola troncata. `label()` si attiene a due cifre esadecimali con un punto esclamativo iniziale in caso di errore, che è la convenzione seguita dai decodificatori integrati.

```
def columns(self):
    return [("Time", 92), ("Hex", 60),
            ("Binary", 80), ("Status", 70)]

def row(self, frame, ascii=True):
    return ["0x%02X" % frame.value,
            format(frame.value, "%db" % frame.length),
            "OK" if frame.status == OK else "CUT"]

def label(self, frame, ascii=True):
    text = "%02X" % (frame.value & 0xFF)
    return ("!" + text) if frame.status else text
```

Quel file è un decodificatore completo ed elencabile. Rilasciato nella cartella `decoders`, compare nella scheda come "Pulse-width bits" con una casella per la dimensione della parola e un menu a discesa per l'ordine dei bit, e decodifica qualsiasi sorgente mappata sul suo unico ingresso.

La scorciatoia seriale

Non reimplementare il campionamento dei bit

L'esempio pratico campiona i bit da sé perché il suo protocollo è autotemporizzato e diverso da qualsiasi cosa integrata. Per qualsiasi cosa della famiglia seriale asincrona, come RS-232, RS-422/485, MODBUS, MIDI, LIN, XBee e DMX512, non reimplementare il campionamento dei bit. Chiamare la funzione ausiliaria condivisa e testata su corpus `serial_bytes()`, che esegue l'auto-baud, il campionamento al centro del bit e i controlli di parità e bit di stop nell'unico modo comprovato, e costruire il framing sul flusso di byte che restituisce.

```
from tds_decode import Decoder, Param
from tds_decode import serial_bytes, STANDARD_BAUDS

class MyUartDecoder(Decoder):

    NAME = "My UART"
    DESCRIPTION = "Async serial, one wire"
    SOURCES = ["Line"]
    PARAMS = [
        Param("baud", "Bit rate", "choice",
              choices=[(0, "Auto")]
              + [(b, str(b)) for b in STANDARD_BAUDS],
              default=0),
        Param("framing", "Framing", "choice",
              choices=[(f, f) for f in
                       ("8N1", "8E1", "8O1", "8N2")],
              default="8N1"),
    ]

    def decode(self, signals, params):
        sig = signals["Line"]
        frames, err, baud, spb = serial_bytes(
            sig, int(params.get("baud") or 0),
            params.get("framing", "8N1"), inverted=False)
        if err:
            return [], err
        return frames, "%d baud, %.1f smp/bit, %d frames" % (
            baud, spb, len(frames))
```

`serial_bytes(sig, baud, framing, inverted)` restituisce quattro valori: i Frame di byte, una stringa di errore che è vuota in caso di successo, la velocità in baud rilevata e i campioni per bit. Un baud pari a 0 gli chiede di rilevare la velocità. `inverted True` legge un senso elettrico idle-low, active-high come il vero RS-232. Un protocollo di livello superiore come MODBUS o LIN chiama `serial_bytes` per i byte grezzi e poi li raggruppa e li controlla nei propri Frame; leggere `modbus.py` e `lin.py` per questo schema.

Installazione e test

Installazione

Rilasciare e aprire

Copiare il file finito nella cartella decoders scrivibile accanto a tdstoolkit.json e aprire, o riaprire, la scheda Forme d'onda. Il decodificatore compare nell'elenco dei protocolli. Non c'è alcun passaggio di registrazione né alcuna ricompilazione. Mappare una sorgente catturata su ciascuno dei suoi ruoli, impostarne i parametri ed eseguirlo.

NOTA. Se il decodificatore non compare, non è stato importato. Controllare nel registro del programma una riga che nomina il file e l'eccezione. Il caricatore salta un file che non si importa anziché far fallire l'intero elenco, quindi un decodificatore difettoso fallisce in silenzio.

Test

Pilotarlo con un Signal sintetico

Un decodificatore è una funzione pura dei suoi Signal e parametri, quindi si testa senza uno strumento costruendo un Signal da un array di campioni predisposto, chiamando decode() e verificando il risultato. Costruire i campioni in modo che il loro significato sia noto, avvolgerli in un Signal, chiamare auto_threshold(), l'unico passaggio che il framework normalmente svolge, e controllare i Frame.

```
from tds_decode import Signal
from pulsebits import PulseBitsDecoder

def _bit(value, period=40):
    """One pulse-width bit: high for part of the period."""
    high = int(period * (0.66 if value else 0.33))
    return [30000] * high + [-30000] * (period - high)

def demo():
    stream = [-30000] * 20 # idle low lead-in
    for bit in (1, 0, 1, 1, 0, 0, 1, 0): # 0xB2, MSB first
        stream += _bit(bit)
    stream += [-30000] * 20 # idle low tail
    sig = Signal(stream, 1e-6) # 1 us per sample
    sig.auto_threshold() # framework does this
    frames, note = PulseBitsDecoder().decode(
        {"Line": sig}, {"bits": 8, "order": "msb"})
    assert len(frames) == 1, note
    assert frames[0].value == 0xB2, "%02X" % frames[0].value
    print("ok:", note)

if __name__ == "__main__":
    demo()
```

Testare ogni decodificatore

Ogni decodificatore integrato è verificato in questo modo. Il decodificatore RS-232 viene eseguito su un corpus di catture reali; ogni altro decodificatore è verificato con un vettore sintetico in `tdstest.py`, ciascuno con un morso di mutazione corrispondente in `bitecheck.py` che rompe il decodificatore di proposito e conferma che il test se ne accorge. Aggiungere un caso per ogni protocollo che viene aggiunto.



ATTENZIONE. *Una decodifica non testata è una decodifica che andrà alla deriva. Aggiungere un vettore sintetico per ogni decodificatore aggiunto, e un morso di mutazione che dimostri che il vettore è portante, prima di fare affidamento sul risultato.*

Lista di controllo

Prima di farvi affidamento

- Il file è denominato in minuscolo senza spazi e non inizia con un trattino basso.
- La classe imposta NAME, DESCRIPTION, SOURCES e PARAMS, e NAME non è vuoto.
- `decode()` restituisce una tupla (frames, note) in ogni percorso, compresi i percorsi di errore.
- `index` e `end_index` del Frame sono assoluti: `sig.first_index` più l'indice di campione locale.
- Un record illeggibile restituisce un elenco vuoto e una nota esplicativa, mai un'eccezione.
- `columns()`, `row()` e `label()` concordano sul numero e sull'ordine dei campi.
- Un test sintetico pilota il decodificatore e verifica i valori decodificati.